

PROPUESTA TÉCNICA Y ARQUITECTURA DE SISTEMA

Proyecto: Plataforma de Auditoría de Código en Tiempo Real con IA

1. INTRODUCCIÓN Y JUSTIFICACIÓN DE LA PROPUESTA

El objetivo de este documento es presentar la arquitectura de software basada en **microservicios políglotas** para el desarrollo de la Plataforma de Auditoría de Código.

Para garantizar la viabilidad económica del proyecto escolar/universitario sin comprometer la robustez empresarial requerida, se ha diseñado una **estrategia Multi-Cloud de capa gratuita (0 USD)**. Esta estructura distribuye la carga de procesamiento, almacenamiento e interfaz en proveedores especializados, eliminando el riesgo de parates por consumo de créditos mensuales.

2. EL STACK TECNOLÓGICO Y DISTRIBUCIÓN CLOUD

La arquitectura se compone de 5 piezas interconectadas estratégicamente:

- **Frontend (Interfaz de Usuario): React** (desplegado en **Vercel**) o **Streamlit** (desplegado en **Streamlit Cloud**). Ambos entornos ofrecen despliegue continuo gratuito y óptimo rendimiento para el cliente.
- **Orquestador Backend (Lógica de Negocio): Java (Spring Boot / Jakarta EE)** desplegado en **Render** (Capa Gratis - Web Service). Se encarga de la seguridad, control de usuarios y auditorías.
- **Analizador de IA (Procesamiento Intensivo): Python (FastAPI)** desplegado en **Koyeb** (Capa Gratis - Nano Instance). Encargado de la comunicación directa con el modelo de lenguaje (LLM).
- **Base de Datos Relacional: PostgreSQL** alojado en **Supabase** (Capa Gratis). Almacena de forma rígida los usuarios, credenciales y cabeceras de auditorías.
- **Base de Datos No Relacional: MongoDB** alojado en **MongoDB Atlas** (Capa Gratis). Almacena los flujos dinámicos de los chats pedagógicos posteriores al análisis.

3. INGENIERÍA DE REPOSITORIOS (ESTRATEGIA GIT)

El proyecto se dividirá estrictamente en **dos (2) repositorios en GitHub**. Esto equilibra el orden conceptual con la simplicidad de despliegue:

Repo 1: Front

- **Contenido:** Código fuente exclusivo de la interfaz (React o Streamlit).
- **Justificación:** Permite a Vercel o Streamlit Cloud leer la raíz del proyecto directamente, facilitando el *CI/CD* (despliegue automático) sin configuraciones complejas.

Repo 2: Back

- **Contenido:** Los dos microservicios separados por carpetas raíz.
- **Justificación:** Centraliza el "contrato de datos" (JSON) en un solo lugar. Render y Koyeb permiten apuntar al mismo repositorio de GitHub configurando el **Root Directory** en sus paneles (*/java-orchestrator* y */python-analyzer* respectivamente).

4. COMUNICACIÓN ENTRE SERVICIOS (EL CONTRATO JSON)

Al estar distribuidos en nubes diferentes, todos los servicios se comunicarán de manera **síncrona mediante el protocolo HTTP (REST API)**, utilizando **JSON** como único formato de transferencia de datos.

El Flujo de Login (El Paso Cero)

Antes de que un alumno pueda auditar código, debe iniciar sesión. Ese proceso sigue este camino:

1. Front -> Java (Render):Credenciales.

El usuario ingresa su email y contraseña en la pantalla de Login (React/Streamlit) y se envía un **POST** a Java (*/api/auth/login*).

2. Java -> Supabase (Postgres):Validación ORM.

Java recibe las credenciales. Mediante el ORM, busca al usuario en la base de datos de **Supabase** por su email y compara la contraseña **HASHEADA**.

3. Generación del JWT (Java):Firma Digital.

Si las credenciales son correctas, el microservicio de Java genera un token **JWT (JSON Web Token)** firmado digitalmente. Este token contiene los datos del usuario (id, username, rol) y una fecha de expiración.

4. Java -> Front:Almacenamiento Local.

Java responde con un código **200 OK** y le devuelve el token JWT al Frontend. El Frontend (React o Streamlit) guarda este token en la memoria del navegador (**localStorage** o sesión).

¿Cómo se conecta el Login con el flujo de auditoría que ya armamos?

Una vez que el usuario ya se logueó y tiene su token guardado en el navegador, el flujo de "Petición Inicial" cambia ligeramente para incluir la seguridad:

1. **Petición con Candado:** Cuando el alumno le da a "Auditar", el Frontend envía el código, pero **obligatoriamente adjunta el token JWT en la cabecera de la petición HTTP** (usando el estándar).
2. **Validación en Java:** Al recibir la petición en Render, Java **no necesita ir a Supabase a validar quién es el usuario**. Gracias a la arquitectura de JWT, Java simplemente descifra el token localmente con su clave secreta. Si el token no expiró y la firma es válida, Java dice: *"Ok, este es Fulano, su ID es 5 y es un alumno válido"*.
3. **Registro de Auditoría:** Una vez validado, Java usa ese ID de usuario extraído del token para hacer el insert del registro inicial en **Supabase (user_id = 5)** y continúa el viaje hacia Python en Koyeb.

Flujo de la Información (Paso a Paso)

1. **Petición Inicial:** El cliente (React/Streamlit) envía el fragmento de código mediante un **POST** con un JSON básico a la URL pública de Java en **Render**:
2. **Orquestación y Seguridad:** Java recibe el JSON, valida la sesión del usuario mediante tokens JWT contra **Supabase**, inserta el registro inicial en Postgres y reenvía el código mediante un **POST** a la URL pública de Python en **Koyeb**.
3. **Inferencia de IA:** Python recibe la petición de Java, interactúa con el LLM (configurado como *Senior Developer*), procesa la respuesta y le devuelve a Java el JSON estructurado con los hallazgos categorizados por severidad:
4. **Persistencia y Cierre:** Java toma este JSON de respuesta, guarda los detalles en **Supabase**, inicializa el hilo de conversación en **MongoDB Atlas** para futuras dudas del alumno y le retorna el JSON final al Frontend para su renderizado en pantalla.

5. REGLAS DE TRABAJO EN EQUIPO Y DESAFÍOS TÉCNICOS

Para asegurar el éxito del desarrollo con un equipo de 5 integrantes, se establecen las siguientes directrices:

- **Política de Branching (Git):** Queda terminantemente prohibido pushear directamente a la rama **main**. Cada integrante trabajará en una rama **feature/nombre-tarea**. Para unificar el código a **main**, se requerirá un *Pull Request (PR)* con la aprobación de al menos otro compañero. De ser posible, trabajar con una rama en paralelo a main (develop) en la que se junten todas las funcionalidades antes de hacer un merge a main.
- **Gestión del "Cold Start" (Desafío Técnico):** Al utilizar las capas gratuitas de Render y Koyeb, los contenedores entrarán en modo de reposo tras 15 minutos de inactividad. **Mitigación:** Durante el desarrollo y 5 minutos antes de la defensa final frente a los profesores, se deberá realizar una petición "despertador" para activar los servidores y asegurar respuestas instantáneas.

- **Variables de Entorno:** Ninguna credencial (claves de base de datos o API keys de IA) debe estar hardcodeada en el código. Se utilizarán archivos `.env` o las secciones de *Environment Variables* de cada nube.

6. Bases de datos

¿Qué guardas en la Base de Datos Relacional (Supabase / Postgres)?

En Postgres guardas únicamente los datos **fijos, estructurados y medibles** de la auditoría. Es la información "dura" que le sirve a la universidad (o a un panel de control de profesores) para sacar estadísticas.

Guardas:

- **La cabecera de la auditoría:** Quién la hizo (`user_id`), el código original que pegó el alumno, el lenguaje (`python`, `java`, etc.), la fecha y el puntaje general (ej: **score: 75/100**).
- **El detalle indexado de los errores:** Una fila por cada hallazgo de la IA, especificando su **severity** (Crítico, Advertencia, Sugerencia), la categoría (Seguridad, Clean Code) y el título del error.

¿Por qué va en la relacional?

Porque si el día de mañana un profesor quiere saber: "*¿Cuántos errores Críticos de Seguridad cometió el alumno Martín en el último mes?*", Postgres resuelve esa consulta (**`SELECT COUNT(*) FROM audit_details WHERE ...`**) en milisegundos gracias a su estructura rígida e índices.

2. ¿Qué guardas en la Base de Datos No Relacional (MongoDB Atlas)?

En Mongo guardas el **hilo de conversación dinámico (el Chat)** que ocurre *después* de que la IA entregó el primer análisis.

Cuando el alumno ve que tiene un error "Crítico por SQL Injection" y usa el chat de Streamlit/React para preguntarle a la IA: "*¿Por qué mi código es inseguro si yo igual valido los datos?*" o "*¿Me das otro ejemplo de cómo arreglarlo?*", todo ese ida y vuelta es un texto libre y dinámico que no entra cómodamente en una tabla SQL.

```

{
  "audit_id": 145, // <--- EL VÍNCULO CON POSTGRES
  "messages": [
    {
      "role": "user",
      "content": "¿Por qué la inyección SQL es peligrosa en este método?",
      "timestamp": "2026-05-30T13:12:00Z"
    },
    {
      "role": "assistant",
      "content": "Es peligrosa porque al concatenar strings...",
      "timestamp": "2026-05-30T13:12:05Z"
    }
  ]
}

```

¿Cómo se conectan ambas bases de datos? (El Vínculo)

El puente de unión es el **audit_id**.

1. Cuando el alumno le da a "Auditar", Java genera una nueva fila en Postgres y el motor le asigna, por ejemplo, el **id: 145**.
2. Java manda el código a Python, Python responde, y Java guarda los errores fijos en Postgres bajo el **audit_id: 145**.
3. En ese mismo instante, Java crea un documento en **MongoDB** con el campo **"audit_id": 145** para inicializar el historial del chat.
4. Cuando el alumno vuelve a entrar a la plataforma y quiere ver su historial, el frontend le pide a Java la auditoría **145**. Java hace un **findById(145)** en Postgres para traer el código y los errores, y hace un **findByAuditId(145)** en Mongo para traer la conversación guardada. El frontend junta todo y lo muestra en una pestaña hermosa.

Resumen de persistencia:

- **Capa Relacional (Supabase/PostgreSQL):** Almacenará datos transaccionales y de auditoría estricta (**Users, Audits, AuditDetails**). Está optimizada para métricas académicas, control de severidad y búsquedas indexadas.
- **Capa No Relacional (MongoDB Atlas):** Almacenará documentos de estructura flexible correspondientes al log conversacional pedagógico continuo entre el alumno y la IA, vinculados de forma unívoca mediante la clave foránea **audit_id** proveniente de la base de datos relacional.

El "Plan de Contingencia" para la API Keys del LLM

Dado que el microservicio de Python en **Koyeb** se conectará a un LLM en la nube (como OpenAI o Gemini) porque no se puede correr un Ollama local directamente dentro de los

servidores gratuitos de Koyeb, el equipo debe saber qué pasa si se quedan sin saldo o cuota gratuita en la API Key de IA.

- **Mitigación técnica:** Detalla en el informe que el código de Python se diseñará de forma **desacoplada**. Si la API Key de OpenAI expira, el encargado de Python solo tendrá que cambiar una variable de entorno para usar la API de Google Gemini o la de Anthropic (Claude), sin tener que reescribir el código de FastAPI.

APÉNDICE: DISEÑO DETALLADO DE BASES DE DATOS

1. CAPA RELACIONAL (PostgreSQL en Supabase)

Esta base de datos maneja la estructura rígida, transaccional y métricas del sistema.

Users

Almacena las credenciales y perfiles de los usuarios que acceden al sistema.

- id BIGINT [PK, Auto-increment]: Identificador único del usuario.
- username VARCHAR(50) [Unique, Not Null]: Nombre de usuario para mostrar en la interfaz.
- email VARCHAR(100) [Unique, Not Null]: Correo electrónico (utilizado para el login).
- password_hash VARCHAR(255) [Not Null]: Contraseña encriptada mediante algoritmo de hasheo.
- role VARCHAR(20) [Not Null]: Rol del usuario (Restringido a: 'STUDENT', 'TEACHER').
- created_at TIMESTAMP [Default: CURRENT_TIMESTAMP]: Fecha y hora de registro.

Audits

Registra la cabecera de cada petición de análisis de código enviada por un alumno.

- id BIGINT [PK, Auto-increment]: Identificador único de la auditoría.
- user_id BIGINT [FK -> users.id, On Delete Cascade, Not Null]: El alumno que solicita la auditoría.
- code_snippet TEXT [Not Null]: El bloque de código fuente original copiado por el alumno.
- language VARCHAR(30) [Not Null]: El lenguaje analizado (ej: 'python', 'java', 'kotlin').
- score_general INT: Puntaje de calidad global asignado por la IA (0 a 100).
- created_at TIMESTAMP [Default: CURRENT_TIMESTAMP]: Fecha y hora de la consulta.

Audits_Details

Almacena de forma indexada cada uno de los hallazgos individuales que la IA detectó en el código. Posee una relación 1 a Muchos (1:N) con la tabla audits.

- id BIGINT [PK, Auto-increment]: Identificador único del hallazgo.
- audit_id BIGINT [FK -> audits.id, On Delete Cascade, Not Null]: Relación con la auditoría madre.
- severity VARCHAR(20) [Not Null]: Severidad del error (Restringido a: 'Critico', 'Advertencia', 'Sugerencia').
- category VARCHAR(50) [Not Null]: Tipo de hallazgo (ej: 'Seguridad', 'Clean Code', 'Sintaxis').
- title VARCHAR(150) [Not Null]: Título descriptivo corto (ej: "Uso de consultas SQL inseguras").
- description TEXT [Not Null]: Resumen del problema detectado.
- affected_lines VARCHAR(50): Líneas específicas donde está el error (ej: "12,14").
- suggested_fix TEXT: El fragmento de código optimizado o corregido propuesto por la IA.
- pedagogical_explanation TEXT: El desarrollo conceptual didáctico del porqué está mal o es ineficiente.

2. CAPA NO RELACIONAL (MongoDB Atlas)

Esta base de datos almacena documentos con estructura flexible para retener el historial conversacional dinámico.

Colección: chat_histories

Cada documento representa el hilo completo de conversación posterior a un análisis de código específico. Se vincula con la base de datos relacional mediante el ID transaccional.

Estructura del Documento (JSON / BSON):

```
{
  "_id": "ObjectId",           // ID único generado por MongoDB
  "audit_id": "Long",          // FK lógica -> Vincula con el ID de la tabla `audits` en PostgreSQL
  "updated_at": "ISODate",     // Última vez que se envió un mensaje en este chat
  "messages": [                // Array embebido con el historial cronológico del chat
    {
      "role": "String",        // Quién envía el mensaje: 'user' (alumno) o 'assistant' (IA)
      "content": "String",     // El contenido de la pregunta o explicación didáctica
      "timestamp": "ISODate"   // Fecha y hora exacta del mensaje
    }
  ]
}
```

RESUMEN DE RELACIONES PARA EL DER (Diagrama Entidad-Relación)

1. **users -> audits (1:N):** Un usuario puede realizar múltiples auditorías a lo largo del cuatrimestre. Una auditoría pertenece obligatoriamente a un único usuario.

2. **audits -> audit_details (1:N)**: Una auditoría de código puede arrojar cero, uno o muchos hallazgos/detalles (por ejemplo, un código puede tener dos advertencias de Clean Code y un error crítico de seguridad).
3. **audits [Postgres] -> chat_histories [MongoDB] (1:1 Lógico)**: Para evitar sobrecargar la base de datos relacional con texto infinito de chats, cada fila en la tabla **audits** se vincula con un único documento de historial de chat en MongoDB a través del campo **audit_id**.